Caso de estudo: construir um pipeline reprodutível de dados e Machine Learning

Modalidade: grupos de 2 a 3 estudantes · **Duração:** 2 semanas · **Entrega:** relatório (máx. 6 páginas), código (`.py` ou `.ipynb`) com `README` e `requirements.txt`, e apresentação de 7 minutos.

Objetivo

Avaliar a maturidade e o conhecimento de cada grupo na **conceção de um projeto de IA**: formular o problema, planear o trabalho, aplicar o CRISP-DM e representar e implementar um pipeline de dados e Machine Learning. Não se pretende um modelo sofisticado: pretende-se um projeto **simples, bem pensado e reprodutível**.

1. Problema e dados

Cada grupo escolhe um dataset público em CSV (≥ 500 registos) e um problema de negócio que o mesmo permita apoiar. Sugestões: **Telco Customer Churn** ou **Bank Marketing** (classificação); **California Housing** ou **Wine Quality** (regressão).

2. Parte teórica: conceção do projeto

- **Planeamento:** ficha de projeto (problema, utilizador, decisão apoiada, variável-alvo, critério de sucesso e o que fica fora do âmbito); papéis dos elementos da equipa; plano em 3 mini-sprints (-Planeamento e desenho; -Desenvolvimento do núcleo functional; - Integração, validação e apresentação), com o entregável de cada uma; 3 riscos principais com a respetiva mitigação.
- **CRISP-DM:** para cada uma das 6 fases (compreensão do negócio, compreensão dos dados, preparação, modelação, avaliação e implementação), um parágrafo com o que foi feito e a decisão tomada.
- **Representação do pipeline:** diagrama do fluxo dados brutos → limpeza → divisão treino/teste → pré-processamento → modelo → avaliação → modelo guardado, indicando onde se evita a fuga de informação.

3. Parte prática: implementação do pipeline

Em Python (pandas e scikit-learn), implementar o pipeline representado:

- carregar os dados sem alterar o ficheiro original; tratar valores omissos e retirar variáveis irrelevantes (por exemplo, identificadores);
- dividir em treino e teste (80/20, `random_state=42`; `stratify` na classificação) **antes** de aprender qualquer transformação;
- construir um `Pipeline` com `ColumnTransformer` (imputação, codificação e normalização) e **um único modelo**: regressão logística (classificação) ou regressão linear (regressão);
- comparar com um baseline simples (`DummyClassifier` ou `DummyRegressor`);
- avaliar no teste: matriz de confusão, precision, recall e F1 (classificação), ou MAE, RMSE e R^2 (regressão);
- interpretar os 3 coeficientes mais influentes e guardar o pipeline completo com `joblib`.

O código deve correr do princípio ao fim sem intervenção manual.

O uso de IA generativa é permitido, mas tem de ser declarado no relatório e verificado criticamente.